

SqueezeLLM: Dense-and-Sparse Quantization

SqueezeLLM: Dense-and-Sparse Quantization

Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, Kurt Keutzer (2024)

메모리 대역폭을 줄이면서 민감한 weight와 outlier를 보존하는 weight-only PTQ

논문의 범위를 먼저 고정한다

속도 논리는 single-batch generative decoding의 matrix-vector 연산에 맞춰져 있다

축	논문 범위	범위 밖 해석
대상	LLM weight matrix	Activation quantization이 아님
방식	Post-training quantization	Full retraining이 아님
실행	Single-batch token decoding	Large-batch GEMM 일반론이 아님
정밀도	3/4-bit index와 FP16 arithmetic	Integer-only inference가 아님
목표	Memory traffic와 model size 감소	연산량 감소만의 문제가 아님

SqueezeLLM은 저비트 계산보다 저비트 저장과 전송을 우선한다

Token decoding은 weight를 계속 다시 읽는다

Batch 1에서는 큰 matrix W 가 token activation vector x 하나에만 사용된다

01	Activation x	현재 token의 FP16 vector
↓		
02	Load W	Layer의 큰 weight matrix를 memory에서 읽음
↓		
03	Matrix-vector	한 vector에 각 weight를 한 번 사용
↓		
04	Output y	다음 연산으로 보낼 FP16 vector
↓		
05	Next token	다음 decoding step에서 weight를 다시 읽음

Arithmetic unit이 빠르더라도 weight 도착을 기다리면 latency는 bandwidth에 묶인다

Memory-bound와 compute-bound를 구분해야 한다

Quantization speedup은 hardware와 batch가 만드는 arithmetic intensity에 달려 있다

상태	병목	Quantization의 직접 효과	대표 조건
Memory-bound	Weight transfer	읽을 byte 감소	Single-batch GEMV
Compute-bound	Arithmetic throughput	저장 감소만으로 부족	Large-batch GEMM 가능
SqueezeLLM	첫 번째 조건을 목표	추가 FP16 lookup을 허용	논문 kernel과 GPU

비균일 LUT의 추가 연산이 작다는 판단도 memory-bound 조건에서 나온다

3 bit는 representative value 여덟 개를 뜻한다

각 weight는 실제 FP16 값을 직접 저장하지 않고 codebook index를 저장한다

저장 값	Bit pattern	Lookup 결과	의미
Index 0	000	Centroid c_0	첫 representative
Index 1	001	Centroid c_1	둘째 representative
Index 2-6	010-110	Centroid c_2-c_6	중간 representatives
Index 7	111	Centroid c_7	여덟째 representative

성능은 여덟 centroid를 어디에 배치하느냐에 크게 좌우된다

Uniform bin은 제한된 codebook을 낭비할 수 있다

Equal-width partition은 weight density와 loss sensitivity를 보지 않는다

상황	Uniform 선택	문제
Weight가 0 근처에 밀집	전체 range에 같은 간격	빈 구간에도 centroid 사용
Rare extreme 존재	최솟값부터 최댓값까지 덮음	보통 값의 resolution 감소
민감도 차이	같은 거리 오차를 동일 취급	중요 weight를 덜 정확히 보존
Integer arithmetic	고정 scale로 계산 용이	Memory-bound latency의 핵심이 아닐 수 있음

같은 3 bit라도 representative 배치가 다르면 loss perturbation이 달라진다

Non-uniform LUT는 필요한 구간에 centroid를 모은다

간격은 고정되지 않으며 channel별 weight와 sensitivity가 학습 위치를 정한다

방식	Centroid 배치	최적화 기준	Runtime 표현
Uniform	Range에 같은 간격	Raw squared error 또는 scale	정수 code와 scale
Unweighted k-means	Weight가 많은 곳에 밀집	Weight reconstruction error	Index와 scalar LUT
SqueezeLLM	민감한 값 쪽으로 더 이동	Fisher-weighted squared error	Index와 FP16 LUT

비균일이라는 말만으로는 부족하며 sensitivity weighting이 핵심 차이다

Weight distance를 final-loss perturbation으로 바꾼다

Taylor expansion에서 출발해 계산 가능한 diagonal Fisher weighted k-means로 단순화한다

01	Quantize W	Original W 를 codebook value WQ 로 교체
↓		
02	Loss change	Final model loss의 변화량을 Taylor 근사
↓		
03	Zero gradient	Converged model 가정으로 linear term 제거
↓		
04	Fisher proxy	Sample gradient outer product로 Hessian 근사
↓		
05	Diagonal	Cross-weight interaction을 버리고 weighted error 계산

Full Hessian을 저장하거나 exact global loss를 직접 최소화하는 방법은 아니다

같은 오차 거리도 sensitivity가 크면 더 비싸다

목표는 각 weight의 squared error에 diagonal Fisher F_{ii} 를 곱해 합하는 것이다

항	역할	큰 값일 때
w_i	Original scalar weight	현재 저장할 대상
$Q(w_i)$	배정된 LUT centroid	w_i 와 가까워야 함
거리 제곱	Quantization magnitude error	Raw reconstruction 손실 증가
F_{ii}	Final loss sensitivity proxy	같은 거리 오차에 더 큰 penalty
Weighted sum	모든 scalar penalty 합	k-means가 줄이는 objective

Sensitivity는 bit를 직접 늘리는 값이 아니라 centroid 배치를 움직이는 weight다

교육용 예시: 네 weight와 두 centroid만 사용한다

논문 실험값이 아니라 sensitivity weighting의 효과를 다시 계산하기 위한 작은 문제다

Weight	-0.20	-0.10	0.10	0.20
Fisher weight	1	1	20	1
민감도 해석	낮음	낮음	매우 높음	낮음

0.10의 quantization error만 다른 값보다 20배 크게 센다

교육용 예시: 민감한 0.10에 centroid를 배정한다

Candidate B는 0.10을 exact하게 표현해 weighted error를 0.0575에서 0.0150으로 낮춘다

Codebook	Centroids	0.10의 error	전체 weighted error
A	{-0.15, 0.15}	$20 \times 0.05^2 = 0.0500$	0.0575
B	{-0.15, 0.10}	$20 \times 0^2 = 0$	0.0150
차이	둘째 centroid 이동	민감한 값 exact	약 74% 감소

B는 raw 최대 오차를 고르게 하지 않고 loss proxy가 큰 값을 우선한다

각 output channel은 자체 scalar LUT를 갖는다

한 channel의 dense weight는 3/4-bit index로 저장되고 필요할 때 FP16 centroid로 복원된다

01	Channel weights	한 output row의 dense scalar values
↓		
02	Weighted k-means	Channel별 8개 또는 16개 centroid 학습
↓		
03	Packed indices	각 scalar를 3/4-bit code로 저장
↓		
04	FP16 LUT	Code가 가리키는 scalar centroid 보관
↓		
05	Piecewise decode	Kernel이 tile별로 읽고 즉시 multiply-add

SqueezeLLM은 vector quantization이 아니라 per-channel scalar codebook 방식이다

저장 정밀도와 계산 정밀도를 구분한다

낮은 bit는 dense weight index에 적용되고 centroid·activation·arithmetic은 FP16이다

구성	정밀도·형식	Runtime 역할
Dense weight	Packed 3/4-bit index	Memory에서 compact하게 load
Dense codebook	FP16 scalar centroids	Index를 actual value로 decode
Activation	FP16	Decoded dense와 sparse value에 곱함
Sparse exceptions	FP16 values와 CSR metadata	Exact sparse contribution 계산
Arithmetic	FP16	DX와 SX를 계산해 합산

3-bit SqueezeLLM을 integer-only inference라고 부르면 구현을 잘못 설명한다

교육용 예시: outlier 하나가 dense range를 늘린다

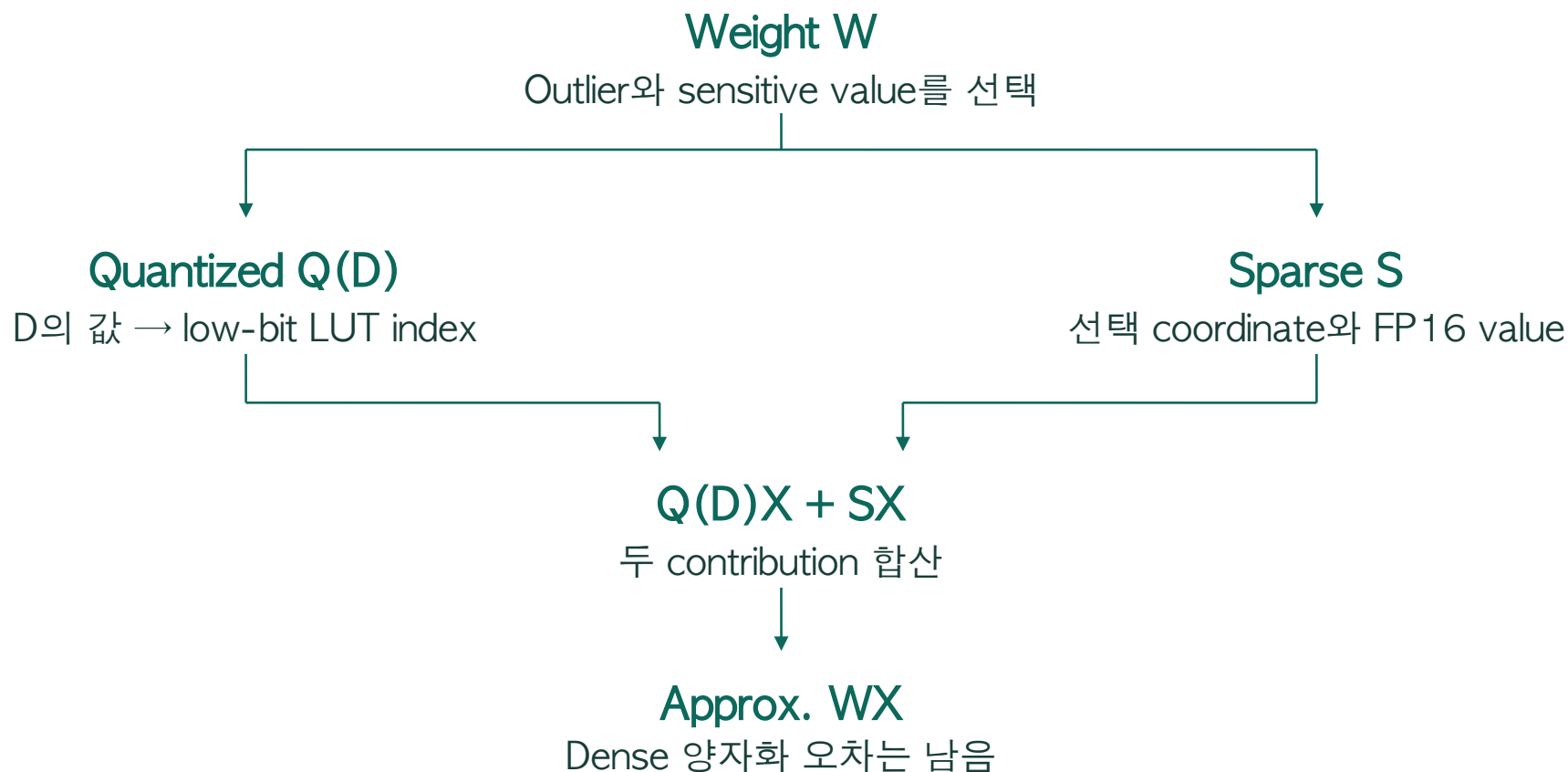
-2.0을 sparse FP16으로 빼면 dense codebook은 -0.20부터 0.20에 집중할 수 있다

표현	값	Dense range	의미
Original W	{-2.0,-0.20,0.10,0.20}	[-2.0,0.20]	Outlier가 range 지배
Dense D	{0,-0.20,0.10,0.20}	[-0.20,0.20]	보통 값 resolution 증가
Sparse S	{-2.0,0,0,0}	해당 없음	위치와 -2.0을 FP16 보존
Reconstruct	$W=D+S$	원래 값 복원	Inference에서 두 product 필요

이 숫자는 threshold나 실제 layer 분포를 재현한 논문 측정값이 아니다

$W=D+S$ 로 분리한 뒤, D만 양자화한다

분리 자체는 정확하지만, 실제 계산 $Q(D)X+SX$ 에는 dense 양자화 오차가 남는다



Sparse S에는 두 종류의 individual value가 들어간다

범위를 넓히는 outlier와 loss proxy가 큰 sensitive weight를 서로 다른 이유로 보존한다

종류	선택 기준	Sparse로 옮기는 이유
Outlier value	Weight percentile threshold 밖	Dense range를 좁힘
Sensitive value	Diagonal Fisher score가 큼	그 값의 quantization error 제거
Centroid 효과	두 종류 모두 dense에서 제거	나머지 centroid가 ordinary value에 집중
보존 단위	Matrix의 individual cell	Whole input-feature column이 아님

S를 outlier-only residual로 설명하면 sensitivity exception을 빠뜨린다

대표 0.45% 설정은 0.05%와 0.4%의 합이다

Sensitive value와 outlier value 비율을 분리해 읽어야 average bit overhead를 이해할 수 있다

구성	전체 weight 비율	저장
Sensitive exceptions	0.05%	FP16 value와 coordinate metadata
Outlier exceptions	0.40%	FP16 value와 CSR metadata
Sparse S 합계	0.45%	Dense index 외 추가 storage
Dense-only	0%	Sparse path 없이 LUT-indexed D만 사용

0.45%는 실험 설정이며 모든 model의 보편 최적 비율이 아니다

Offline 단계는 두 저장 형식을 만든다

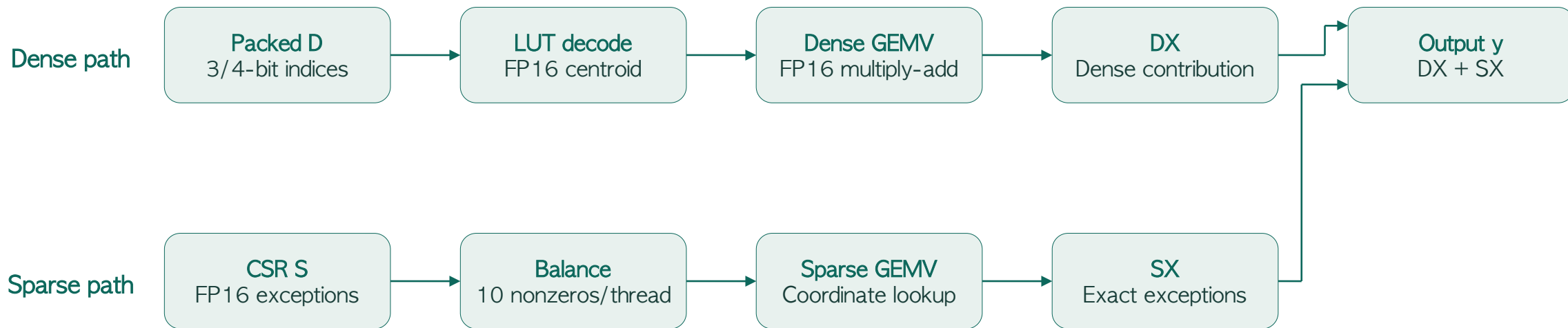
Calibration과 k-means는 배포 전 한 번 수행하며 decoding kernel 안에서 반복하지 않는다

01	Calibration	Sample에서 final loss gradient 계산
↓		
02	Diagonal Fisher	Weight별 sensitivity score 생성
↓		
03	Extract S	Outlier와 top sensitive values 선택
↓		
04	Fit D LUT	남은 value에 channelwise weighted k-means
↓		
05	Pack model	3/4-bit indices, FP16 LUT, CSR exceptions 저장

Offline Fisher-k-means 비용과 online token latency를 같은 시간으로 비교하면 안 된다

Runtime은 dense와 sparse path를 함께 실행한다

두 kernel이 같은 FP16 activation을 사용하고 DX와 SX를 더해 output vector를 만든다



Dense approximation만 계산하면 sparse로 보존한 weight contribution을 잃는다

Dense kernel은 lookup 뒤 바로 누적한다

Packed index를 작은 조각으로 읽고 LUT lookup 뒤 FP16 multiply-add를 수행한다

01	Read bits	Compressed matrix에서 3/4-bit code load
↓		
02	Index LUT	해당 output channel의 FP16 centroid 선택
↓		
03	Dequantize	필요한 weight tile만 FP16 value로 변환
↓		
04	Multiply	FP16 activation element와 곱함
↓		
05	Accumulate	Output channel의 DX에 즉시 누적

저비트 index는 memory traffic을 줄이고 FP16 arithmetic은 기존 compute를 사용한다

Sparse matrix는 작아도 row별 nonzero가 불균형하다

한 thread가 한 row를 맡으면 긴 row를 처리하는 thread가 전체 kernel을 지연시킨다

Row	Nonzeros 예시	One-row-per-thread	Fixed-work 방식
r0	2	빨리 종료	다른 row와 묶어 처리
r1	3	빨리 종료	다른 row와 묶어 처리
r2	30	한 thread에 긴 tail	여러 thread로 분할
r3	1	대부분 대기	정해진 nonzero 수만 담당

Sparsity percentage만 작다고 sparse kernel overhead가 자동으로 작은 것은 아니다

Balanced kernel은 thread마다 nonzero 열 개를 맡긴다

Row boundary를 넘겨 work를 균등화하는 대신 synchronization이 추가된다

01	CSR entries	Row별 길이가 다른 nonzero sequence
↓		
02	Flatten work	Nonzero 기준으로 처리 구간을 나눔
↓		
03	10 per thread	논문 구현의 fixed assignment
↓		
04	Sync rows	한 row를 여러 thread가 처리해 synchronization
↓		
05	Add SX	Balanced partial sums를 output에 누적

균형 개선은 공짜가 아니며 synchronization과 metadata access를 교환한다

Sparse FP16 예외에는 실제 latency와 memory 비용이 있다

LLaMA-7B 3-bit·0.45%·A6000·128-token 조건에서 kernel 선택이 overhead를 좌우했다

Kernel	Sparsity	Latency	Peak memory
Dense only	0%	1.5 s	2.9 GB
Standard CSR	0.45%	3.9 s	3.2 GB
Balanced sparse	0.45%	1.7 s	3.1 GB

Standard CSR의 100% 초과 overhead를 balanced kernel이 약 13%로 낮춘 조건이다

Ablation은 두 아이디어가 각각 기여함을 보여 준다

LLaMA- 7B·3-bit·C4 perplexity에서 uniform부터 dense+sparse까지 단계별로 비교한다

설정	C4 PPL	추가된 요소
RTN uniform	28.26	Equal-width baseline
Non-uniform, no sensitivity	18.08	Unweighted scalar k-means
Sensitivity dense-only	7.75	Diagonal-Fisher weighted k-means
+0.05% sensitive S	7.67	Sensitive exact exceptions
0.45% total S	7.56	0.05% sensitive + 0.4% outlier

가장 큰 개선은 sensitivity weighting이며 sparse 예외가 추가 개선을 만든다

Nominal 3 bit와 average bit width는 다르다

LUT·sparse FP16 values·CSR metadata가 dense index 외 storage를 추가한다

LLaMA-7B 설정	Nominal	Average	Memory
GPTQ no grouping	3 bit	3.00 bit	2.9 GB
SqueezeLLM dense	3 bit	3.02 bit	2.9 GB
GPTQ group 128	3 bit	3.24 bit	3.0 GB
SqueezeLLM 0.45%	3 bit	3.24 bit	3.1 GB

정확도 비교는 같은 nominal label보다 비슷한 model size·average bits로 묶어야 한다

비슷한 size 안에서 quality를 비교한다

LLaMA-7B의 C4·WikiText-2 perplexity는 average width와 함께 읽는다

Method	Avg bits	C4 PPL	Wiki PPL	Memory
GPTQ	3.00	9.55	7.55	2.9 GB
SqueezeLLM dense	3.02	7.75	6.32	2.9 GB
GPTQ g128	3.24	7.89	6.27	3.0 GB
SqueezeLLM 0.45%	3.24	7.56	6.13	3.1 GB

Dense와 sparse variant는 서로 다른 storage point이므로 별도 comparison group이다

Latency 결과에는 GPU와 token 수를 붙여야 한다

A6000에서 LLaMA-7B가 128 tokens를 생성할 때의 저자 kernel 측정값이다

Method	Bit width	Latency	Memory
FP16 baseline	16	3.2 s	12.7 GB
GPTQ	3	1.4 s	2.9 GB
SqueezeLLM dense	3.02	1.5 s	2.9 GB
GPTQ g128	3.25	13.7 s	3.0 GB
SqueezeLLM 0.45%	3.24	1.7 s	3.1 GB

1.7초를 다른 batch·GPU·kernel의 예상 latency로 직접 옮길 수 없다

Grouped GPTQ latency는 구현 조건에 민감하다

논문은 activation ordering의 permutation과 distributed access가 coalescing을 해친다고 설명한다

- 저자들은 공개 GPTQ-For-LLaMA codebase를 바탕으로 비교 kernel을 구현했다
- Grouped GPTQ의 official kernel이 없어 저자 구현 조건에 의존한다
- Activation ordering은 같은 channel element에 다른 scale을 연결할 수 있다
- Permutation과 흩어진 access는 GPU coalesced memory access를 방해한다
- 따라서 13.7초는 GPTQ family 전체의 보편 latency가 아니다

Hardware-specific kernel 결과를 algorithm 자체의 절대 속도 순위로 일반화하지 않는다

Sensitivity는 calibration sample에서 계산한다

Main setup은 model family에 맞는 training set에서 random sample 100개를 사용한다

대상	Calibration source	Sample 수
Vicuna	Vicuna training set	100 random samples
그 외 models	C4 training set	100 random samples
LLaMA2-7B 4-bit 추가 실험	C4 calibration	10과 100에서 C4 PPL 7.72
해석	한 configuration의 data efficiency	10개가 항상 충분하다는 보장 아님

Calibration data는 inference input이 아니라 offline Fisher estimator의 표본이다

Quantization 자체에도 memory와 시간이 든다

Fisher는 A100, k-means는 48-core Xeon에서 측정한 offline one-time cost다

Model	Peak memory	Fisher	K-means
LLaMA-7B	33 GB	0.3 min	11 min
LLaMA-13B	61 GB	0.6 min	17 min
LLaMA-30B	149 GB	1.3 min	45 min
LLaMA-65B	292 GB	2.5 min	80 min

배포 model은 작아지지만 quantization workstation 요구량은 별도다

보존 단위를 보면 인접 방법과 구분된다

SqueezeLLM은 scalar LUT와 individual sparse value를 결합한다

Method family	압축 단위	고정밀로 남기는 단위
SqueezeLLM	Per-channel scalar LUT index	Individual sensitive-outlier cells
OWQ	Quantized matrix와 예외	Selected whole input-feature columns
VPTQ류 VQ	Vector-codebook index	Vector representation 중심
Uniform RTN	Equal-width scalar bins	별도 sparse exception 없음
Sparse-only	Dense quantizer와 exceptions	Weighted LUT가 없을 수 있음

D+S만 보거나 non-uniform LUT만 보면 SqueezeLLM proposal의 절반만 본다

논문 결과에서 말할 수 있는 범위를 지킨다

Accuracy·storage·latency 수치는 각각 비교 budget과 measurement condition이 있다

말할 수 있음	말하면 안 됨
LLaMA-7B 3-bit C4에서 weighted LUT가 ablation 개선	모든 model에서 같은 PPL 감소
0.45% 설정이 quality를 더 개선	Sparse FP16이 cost 없이 exact model을 만듦
A6000·batch 1·128-token에서 speedup 측정	모든 GPU·batch에서 동일 latency
Diagonal Fisher가 practical loss proxy	Full Hessian exact optimization

SqueezeLLM의 강점은 조건을 붙여도 남는 storage-quality-latency trade-off다

남는 한계와 비용

저비트 weight storage를 잘 쓰는 방법이지 모든 inference regime의 해법은 아니다

- Batch-1 GEMV가 중심이며 large-batch는 compute-bound일 수 있다
- Diagonal Fisher는 cross-weight interaction을 무시하는 근사다
- Sparse FP16 values와 CSR metadata가 average bit와 memory를 늘린다
- Skew-aware kernel이 없으면 sparse path runtime이 크게 증가할 수 있다
- Generation 중심이며 encoder·encoder-decoder 평가는 제한적이다
- Roofline hardware model은 inference pipeline을 단순화한 가정을 사용한다

실제 채택 판단에는 target batch·GPU·kernel·quality metric의 재측정이 필요하다

SqueezeLLM을 기억하는 다섯 문장

중요한 weight에는 더 좋은 representative를 주고 예외는 sparse FP16으로 보존한다

01	Memory wall	Batch-1 decoding은 weight traffic이 latency를 지배
↓		
02	Weighted LUT	Diagonal Fisher로 민감한 scalar error를 더 크게 계산
↓		
03	Dense D	나머지 value를 channelwise non-uniform index로 저장
↓		
04	Sparse S	Outlier와 sensitive individual value를 FP16 보존
↓		
05	Two kernels	Packed LUT GEMV와 balanced CSR GEMV를 합산

핵심은 3 bit라는 숫자가 아니라 제한된 memory budget의 배분 방식이다

공개 출처

출판 record·원문 PDF·preprint·공식 구현을 공개 URL에서 확인할 수 있다

자료	용도	공개 위치
PMLR record	ICML 2024 서지 identity	proceedings.mlr.press/v235/kim24f.html
PMLR PDF	본문·표·appendix의 직접 근거	proceedings.mlr.press/v235/kim24f/kim24f.pdf
arXiv	공개 preprint record	arxiv.org/abs/2306.07629
Official code	저자 공개 implementation	github.com/SqueezeAILab/SqueezeLLM

슬라이드 locator는 23-page PMLR PDF의 page 번호를 따른다